LAB 2 – Ethernet, komunikacja TCP i UDP

Przemysłowe urządzenia i protokoły komunikacyjne
Automatyka i Robotyka

1. Krótkie wprowadzenie teoretyczne

Socket to programistyczny punkt komunikacji sieciowej. Aplikacja korzysta z adresu IP i numeru portu, aby wysyłać lub odbierać dane.

TCP jest protokołem połączeniowym. Przed wymianą danych klient zestawia połączenie z serwerem. TCP pilnuje kolejności danych, potwierdzeń i retransmisji utraconych fragmentów. Dobrze nadaje się tam, gdzie ważna jest kompletność danych, np. w Modbus TCP.

UDP jest protokołem bezpołączeniowym. Nadawca wysyła datagram do wskazanego adresu i portu bez wcześniejszego zestawiania sesji. UDP nie gwarantuje dostarczenia, kolejności ani ponownego wysłania utraconych danych. Ma za to mały narzut i prostą obsługę.

Cecha	TCP	UDP
Połączenie	Tak	Nie
Potwierdzenia i retransmisja	Tak	Nie
Kolejność danych	Gwarantowana	Nie jest gwarantowana
Typowe funkcje w Pythonie	connect, listen, accept, send/recv	sendto, recvfrom
Przykładowe zastosowanie	Modbus TCP, HTTP	telemetria, broadcast, szybkie dane cykliczne

Ważne: brak odpowiedzi na ping nie oznacza automatycznie braku komunikacji TCP/UDP. ICMP może być blokowany niezależnie od usług TCP i UDP.

2. Przykład UDP na localhost

Utwórz w PyCharm dwa pliki: **udp_server.py** oraz **udp_client.py**. Najpierw uruchom serwer, następnie klienta.

udp_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5001

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))

print(f"Serwer UDP działa na {HOST}:{PORT}")
print("Oczekiwanie na wiadomości...")

while True:
    data, address = server.recvfrom(1024)
    message = data.decode("utf-8")
    print(f"Odebrano od {address}: {message}")
```

```
    if message.lower() == "exit":
        break

    response = f"ACK: {message}"
    server.sendto(response.encode("utf-8"), address)

server.close()
```

udp_client.py

```
import socket

SERVER_IP = "127.0.0.1"
PORT = 5001

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.settimeout(3)

while True:
    message = input("Wpisz wiadomość: ")
    client.sendto(message.encode("utf-8"), (SERVER_IP, PORT))

    if message.lower() == "exit":
        break

    try:
        data, address = client.recvfrom(1024)
        print("Odpowiedź serwera:", data.decode("utf-8"))
    except socket.timeout:
        print("Brak odpowiedzi serwera.")

client.close()
```

Sprawdź: wyślij kilka różnych wiadomości, np. TEMP=23.5, MOTOR=ON, SPEED=1200. Zobacz, jaki adres i port klienta wyświetla serwer.

3. Przykład TCP na localhost

Utwórz pliki **tcp_server.py** oraz **tcp_client.py**. Ponownie najpierw uruchom serwer.

tcp_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print(f"Serwer TCP nasłuchuje na {HOST}:{PORT}")
conn, addr = server.accept()
print("Połączono z:", addr)
```

```
while True:
    data = conn.recv(1024)
    if not data:
        break

    message = data.decode("utf-8")
    print("Odebrano:", message)

    if message.lower() == "exit":
        break

    conn.sendall(f"ACK: {message}".encode("utf-8"))

conn.close()
server.close()
```

tcp_client.py

```
import socket

SERVER_IP = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))

print("Połączono z serwerem")

while True:
    message = input("Wpisz wiadomość: ")
    client.sendall(message.encode("utf-8"))

    if message.lower() == "exit":
        break

    response = client.recv(1024)
    print("Odpowiedź serwera:", response.decode("utf-8"))

client.close()
```

4. Zadanie: komunikacja między dwoma stanowiskami

Dobierzcie się w pary. Jeden komputer będzie pełnił rolę serwera, drugi klienta. Wykorzystajcie wiedzę z poprzednich zajęć do znalezienia adresów IPv4 obu komputerów.

1. Na obu komputerach sprawdź adres IPv4, sprawdź czy istnieje połączenie pomiędzy komputerami wykorzystując ping
2. W UDP na komputerze-serwerze zmień HOST z 127.0.0.1 na 0.0.0.0.
3. W kliencie UDP wpisz rzeczywisty adres IPv4 komputera-serwera.
4. Uruchom serwer, potem klienta i prześlij dowolną wiadomość.
5. Powtórz to samo dla TCP: serwer nasłuchuje na 0.0.0.0, klient łączy się z IP sąsiada.
6. Zamieńcie się rolami i sprawdźcie, czy komunikacja działa w drugą stronę.

Nie zakładaj, że ping musi działać. Jeżeli aplikacja komunikuje się poprawnie mimo braku odpowiedzi ICMP, jest to wartościowa obserwacja.

5. Jak zachowują się TCP i UDP?

7. **Uruchom klienta, gdy serwer nie działa.** Porównaj zachowanie TCP i UDP. Czy błąd pojawia się od razu? Czy klient UDP wie, że nikt nie odebrał datagramu?
8. **Ustaw inny port w kliencie niż w serwerze.** Sprawdź różnicę w zachowaniu obu protokołów.
9. **Zatrzymaj serwer podczas działania klienta.** Co zauważa klient TCP? Co dzieje się w UDP?
10. **Wyślij szybko serię kilkunastu wiadomości.** Obserwuj kolejność i sposób odbioru. Nie zakładaj, że pojedyncze recv() odpowiada jednej logicznej wiadomości w TCP.
11. **Uruchom netstat -an przed i po zestawieniu TCP.** Znajdź LISTENING i ESTABLISHED. Porównaj z UDP.

6. Zadanie: zbuduj własny mini-protokół automatyki

Przestań wysyłać dowolny tekst. Ustal prosty format komunikatów związany z automatyką. Przykład:

```
READ:TEMP
READ:SPEED
WRITE:MOTOR:ON
WRITE:MOTOR:OFF
```

Serwer ma interpretować polecenie i wygenerować odpowiedź. Możesz na początku przechowywać wartości w zwykłych zmiennych w Pythonie.

- READ:TEMP -> zwróć temperaturę, np. TEMP=23.7
- READ:SPEED -> zwróć prędkość, np. SPEED=1450
- WRITE:MOTOR:ON -> ustaw stan silnika i odeślij ACK
- nieznane polecenie -> odeślij ERR:UNKNOWN_COMMAND

Cel eksperymentu: zobaczyć, że protokół komunikacyjny to nie tylko transport TCP/UDP, lecz również uzgodniona struktura i znaczenie danych.

7. Zadanie: komunikacja dwukierunkowa

Rozbuduj program tak, aby obie strony aktywnie wymieniały informacje, a odpowiedź zależała od treści żądania.

- Klient może pytać o wartości procesowe.
- Serwer może potwierdzać wykonanie polecenia lub zwracać błąd.
- Dodaj przynajmniej trzy różne komendy i trzy różne odpowiedzi.
- Dodaj prosty status urządzenia, np. MOTOR=ON/OFF i ALARM=0/1.

Przykładowa rozmowa:

```
Klient -> READ:TEMP
Serwer -> TEMP=24.1
```

```
Klient -> WRITE:MOTOR:ON
Serwer -> ACK:MOTOR=ON

Klient -> READ:STATUS
Serwer -> MOTOR=ON;ALARM=0
```

8. Zadanie: obsługa błędów*

Aplikacja przemysłowa nie powinna kończyć działania przy pierwszym problemie. Dodaj obsługę co najmniej trzech sytuacji błędnych.

Lista wbudowanych wyjątków w Python: Built-in Exceptions — Python 3.15.2 documentation

- serwer jest niedostępny albo nie odpowiada w określonym czasie;
- użytkownik wpisał niepoprawne polecenie;
- wiadomość ma niewłaściwy format, np. WRITE:MOTOR bez wartości;
- połączenie TCP zostało zerwane;
- odebrane dane nie mogą zostać zdekodowane lub przetworzone.

Wykorzystaj konstrukcję:

```
try:
    # operacja sieciowa
except socket.timeout:
    print("Przekroczono czas oczekiwania")
except OSError as error:
    print("Błąd komunikacji:", error)
```

Eksperyment: celowo wywołaj każdy z obsługiwanych błędów i sprawdź, czy program zachowuje się przewidywalnie zamiast się kończyć.

9. Zadanie rozszerzone - wielu klientów*

Zbuduj serwer, który obsłuży jednocześnie kilka stanowisk. Każdy klient powinien posiadać własny identyfikator urządzenia, np. SENSOR_1, ROBOT_2, PLC_3.

- dla TCP użyj np. modułu threading albo rozwiązania nieblokującego;
- serwer zapisuje ostatnią wartość otrzymaną od każdego klienta;
- klienci mogą cyklicznie wysyłać dane procesowe;
- serwer wyświetla prostą tabelę: ID urządzenia, adres IP, wartość, czas ostatniej aktualizacji;
- dodaj mechanizm wykrywania braku nowych danych przez określony czas.

Przykład danych:

```
SENSOR_1;TEMP=23.4
ROBOT_2;STATE=RUN
PLC_3;COUNTER=128
```

Możliwe rozszerzenie: klient otrzymuje od serwera komendę sterującą, np. SET:SPEED=1200, i odsyła potwierdzenie.

10. Wnioski do zapisania po zajęciach

- Kiedy wybrałbyś TCP, a kiedy UDP w aplikacji automatyki?
- Która cecha TCP była najbardziej widoczna podczas eksperymentów?
- Dlaczego sam adres IP nie wystarcza do wskazania konkretnej aplikacji?
- Co powinien robić program, gdy urządzenie przestaje odpowiadać?
- Jak własny mini-protokół przypomina rzeczywiste protokoły przemysłowe?

Najważniejszy efekt laboratorium: po zajęciach student powinien umieć otworzyć prosty kod oparty o socket, zmienić adres/port/logikę wiadomości, uruchomić komunikację oraz samodzielnie zdiagnozować podstawowe problemy.